VIDYAVARDHINI'S COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY
K.T. Marg, Vasai Road (W), Dist-Palghar - 401202, Maharashtra

ADV DEVOPS EXPERIMENT-7

Aim : To Build, change, and destroy AWS infrastructure Using Terraform.

Theory :

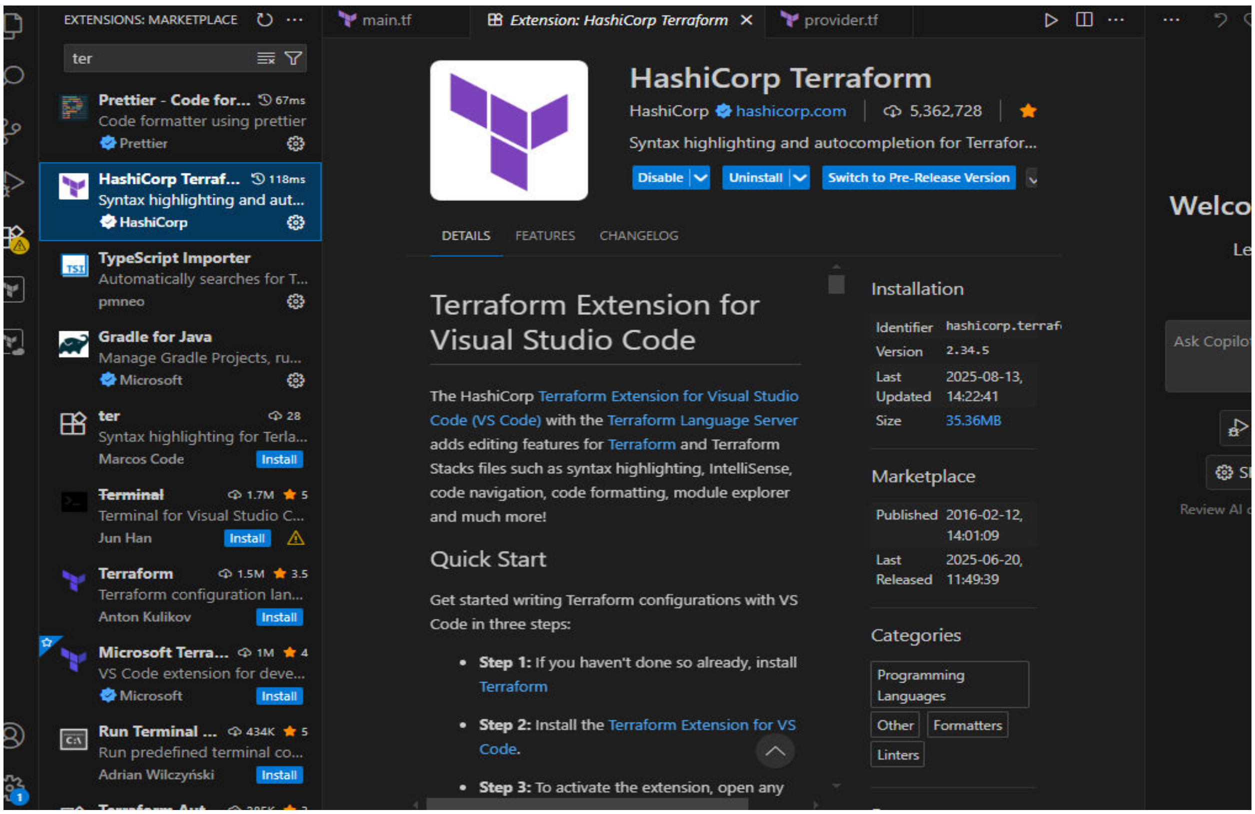
To Build AWS infrastructure using terraform make sure to have Java installed and running on your machine. For instructions, see Java Development kit. Apache maven Java Lambdas are build using mvn packages and are deployed using Terraform into AWS set up terraform. For steps, see Terraform downloads an AWS account.

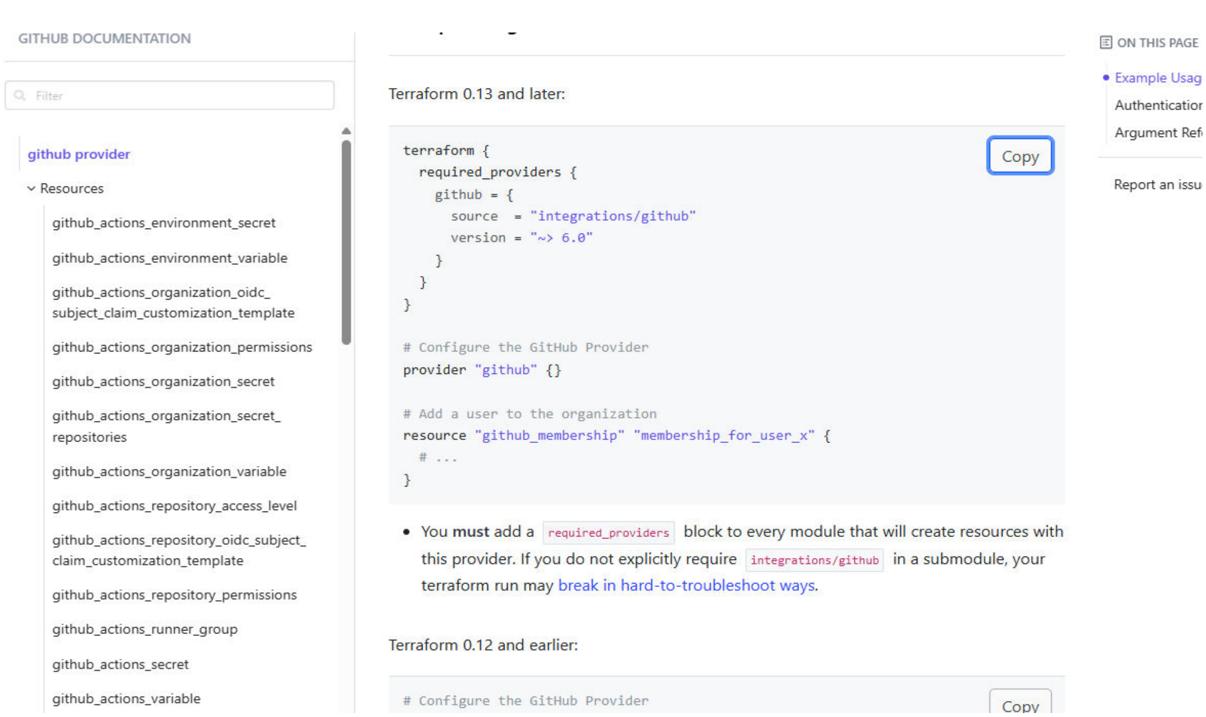
To change AWS infrastructure using terraform you created your first infrastructure with terraform: A single EC2 Instance on AWS. Infrastructure is continuously evolving, and
terraform helps you manage that charge. As you change terraform configurations. Terraform builds an execution plan that only modifies what is necessary to reach you desired state.
To Destroy AWS infrastructure using terraform once you no longer need infrastructure, you may want to destroy it to reduce your security exposure & costs. For example, you may remove a production environment from service, or manage short-lived environments like build or testing systems.
In addition to build and modifying infrastructure, terraform can destroy or recreate the infrastructure it manages.
Hashicrop terraform is an open source infrastructure. As code software tool that allows DevOps engineers to programmatically provision the physical resources an application requires to run.

Lifecycle of terraform is given below.

1. Terraform init
2. Terraform validate
3. Terraform plan.
4. Terraform apply.
5. Terraform destroy.

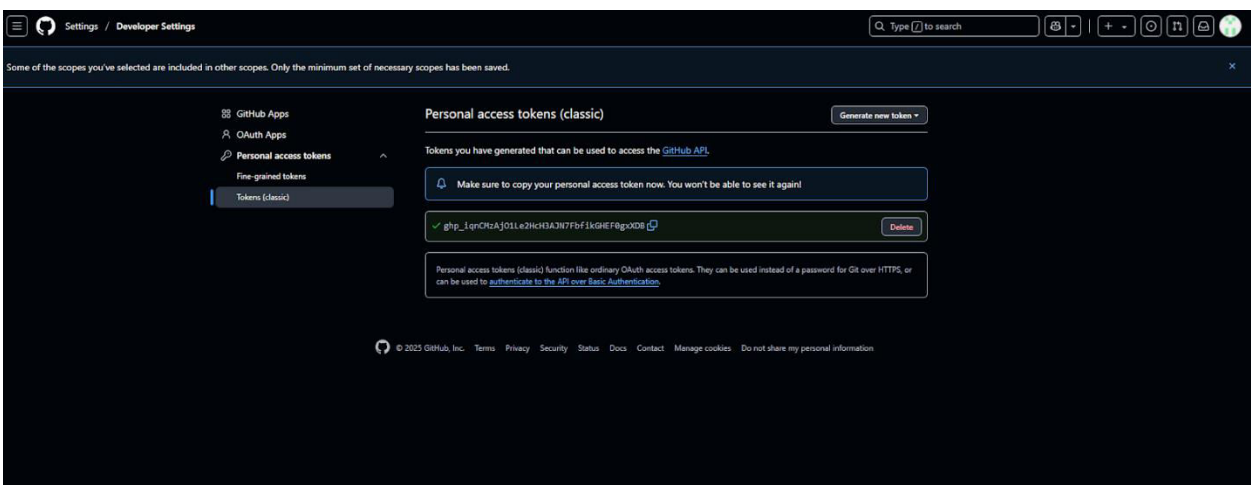
Procedure :

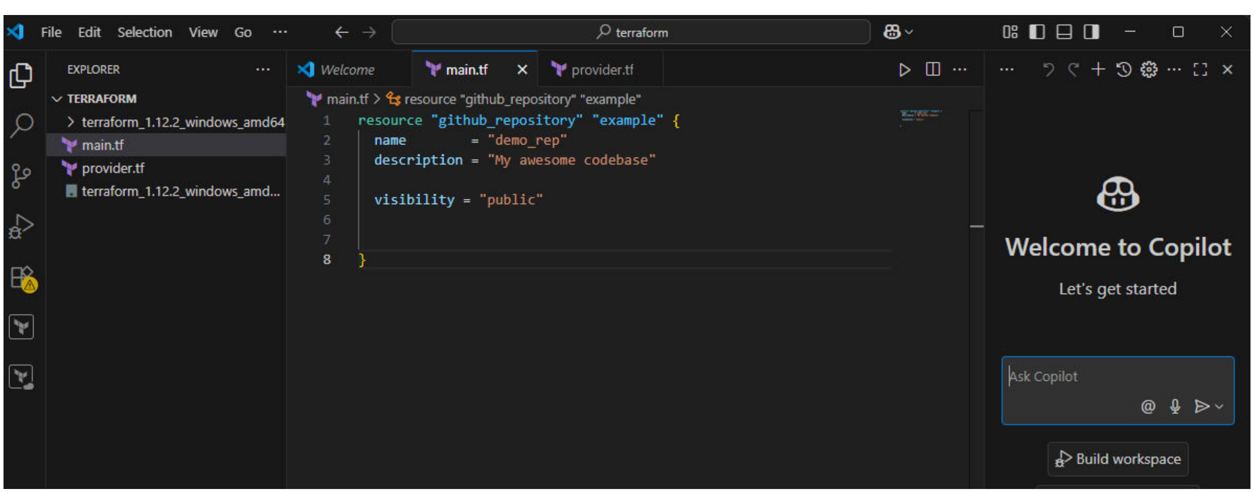

EXTENSIONS: MARKETPLACE
ter
Prettier - Code for... 67ms
Code formatter using prettier
Prettier
HashiCorp Terraf... 118ms
Syntax highlighting and aut...
HashiCorp
TypeScript Importer
Automatically searches for T...
pmneo
Gradle for Java
Manage Gradle Projects, ru...
Microsoft
ter 28
Syntax highlighting for Terla...
Marcos Code
Install
Terminal 1.7M 5
Terminal for Visual Studio C...
Jun Han
Install
Terraform 1.5M 3.5
Terraform configuration lan...
Anton Kulikov
Install
Microsoft Terra... 1M 4
VS Code extension for deve...
Microsoft
Install
Run Terminal ... 434K 5
Run predefined terminal co...
Adrian Wilczyński
Install
main.tf
Extension: HashiCorp Terraform
provider.tf
HashiCorp Terraform
HashiCorp hashicorp.com 5,362,728
Syntax highlighting and autocompletion for Terrafor...
Disable
Uninstall
Switch to Pre-Release Version
DETAILS
FEATURES
CHANGELOG
Terraform Extension for Visual Studio Code
The HashiCorp Terraform Extension for Visual Studio Code (VS Code) with the Terraform Language Server adds editing features for Terraform and Terraform Stacks files such as syntax highlighting, IntelliSense, code navigation, code formatting, module explorer and much more!
Quick Start
Get started writing Terraform configurations with VS Code in three steps:
Step 1: If you haven't done so already, install Terraform
Step 2: Install the Terraform Extension for VS Code.
Step 3: To activate the extension, open any
Installation
Identifier hashicorp.terraf
Version 2.34.5
Last Updated 2025-08-13, 14:22:41
Size 35.36MB
Marketplace
Published 2016-02-12, 14:01:09
Last Released 2025-06-20, 11:49:39
Categories
Programming Languages
Other
Formatters
Linters
Welco
Ask Copilo
Review AI

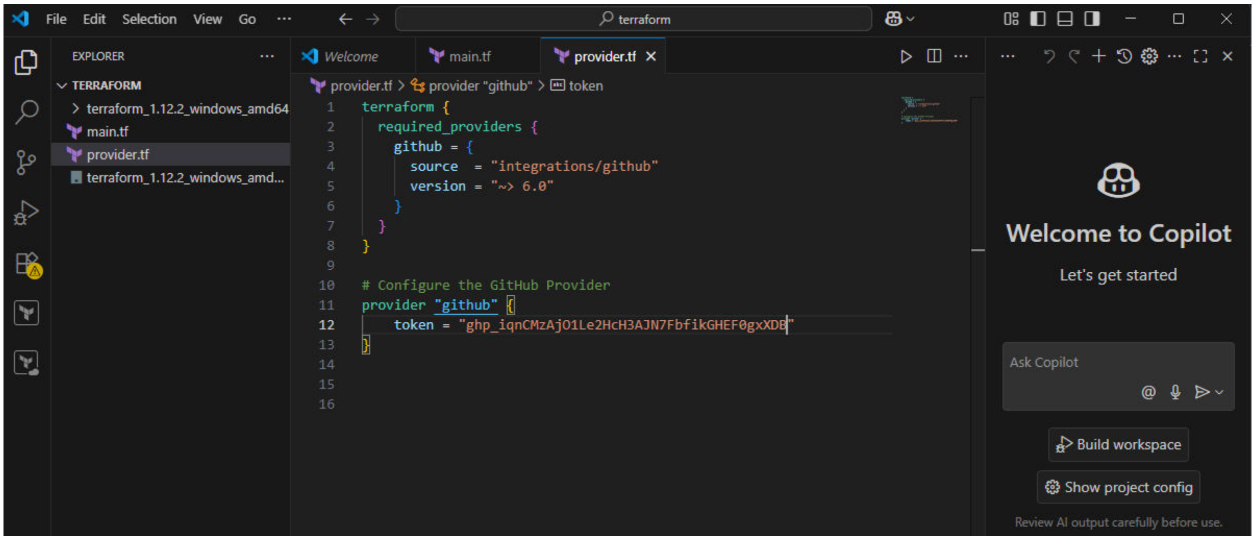

GITHUB DOCUMENTATION
Filter
github provider
Resources
github_actions_environment_secret
github_actions_environment_variable
github_actions_organization_oidc_subject_claim_customization_template
github_actions_organization_permissions
github_actions_organization_secret
github_actions_organization_secret_repositories
github_actions_organization_variable
github_actions_repository_access_level
github_actions_repository_oidc_subject_claim_customization_template
github_actions_repository_permissions
github_actions_runner_group
github_actions_secret
github_actions_variable
Terraform 0.13 and later:
Copy
terraform {
required_providers {
github = {
source = "integrations/github"
version = "~> 6.0"
}
}
}
# Configure the GitHub Provider
provider "github" {}
# Add a user to the organization
resource "github_membership" "membership_for_user_x" {
# ...
}
You must add a required_providers block to every module that will create resources with this provider. If you do not explicitly require integrations/github in a submodule, your terraform run may break in hard-to-troubleshoot ways.
Terraform 0.12 and earlier:
Configure the GitHub Provider
Copy
ON THIS PAGE
Example Usag
Authenticatior
Argument Ref
Report an issu

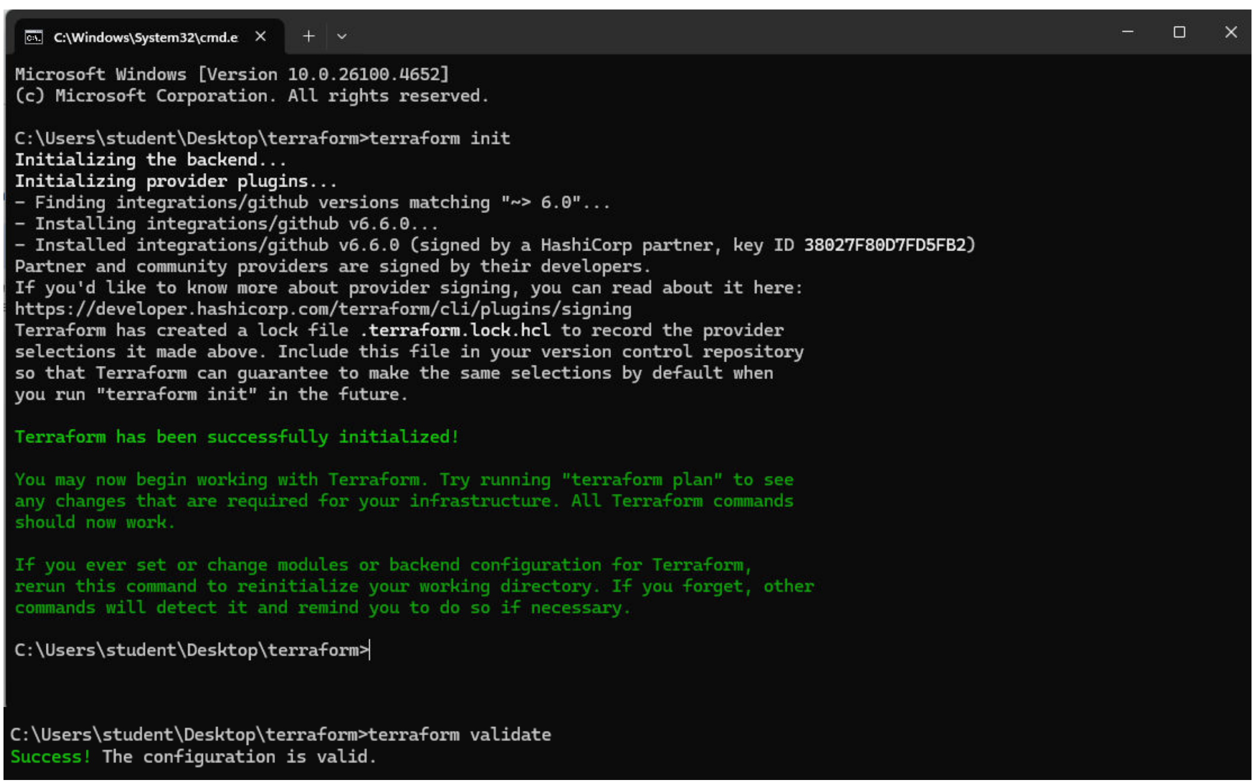
Example Usage

```
resource "github_repository" "example" {
  name        = "example"
  description = "My awesome codebase"

  visibility = "public"

  template {
    owner                = "github"
    repository           = "terraform-template-module"
    include_all_branches = true
  }
}
```



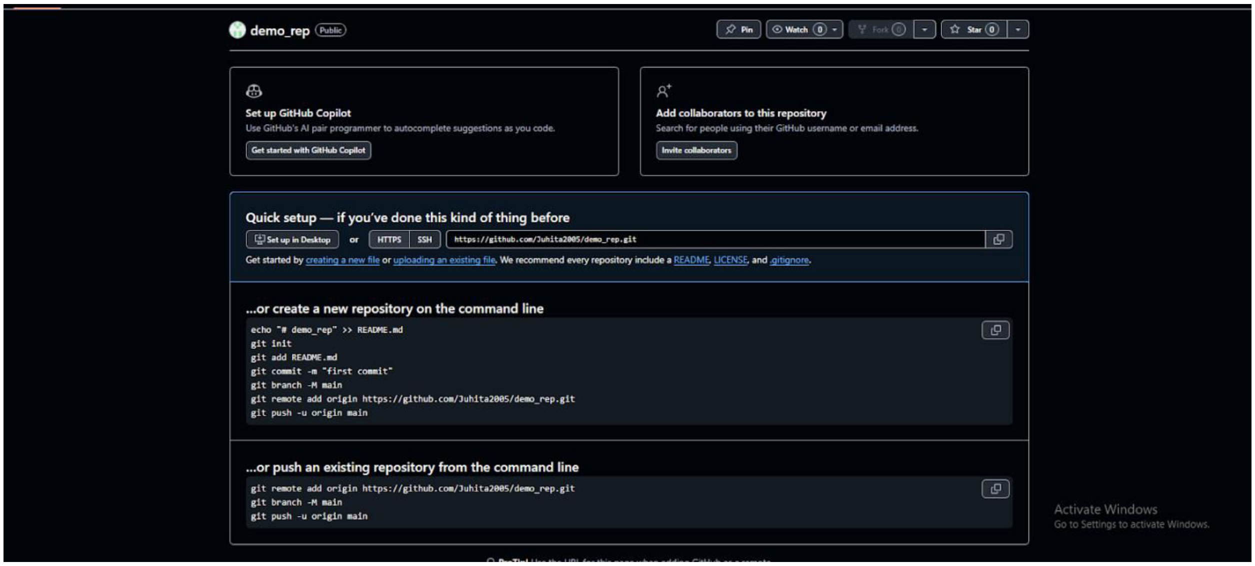



File Edit Selection View Go ...
terraform
EXPLORER
TERRAFORM
terraform_1.12.2_windows_amd64
main.tf
provider.tf
terraform_1.12.2_windows_amd...
Welcome
main.tf
provider.tf
provider.tf > provider "github" > token
terraform {
required_providers {
github = {
source = "integrations/github"
version = "~> 6.0"
}
}
}
# Configure the GitHub Provider
provider "github" {
token = "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"
}
Welcome to Copilot
Let's get started
Ask Copilot
Build workspace
Show project config
Review AI output carefully before use.


C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26100.4652]
(c) Microsoft Corporation. All rights reserved.
C:\Users\student\Desktop\terraform>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding integrations/github versions matching "~> 6.0"...
- Installing integrations/github v6.6.0...
- Installed integrations/github v6.6.0 (signed by a HashiCorp partner, key ID 38027F80D7FD5FB2)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
C:\Users\student\Desktop\terraform>
C:\Users\student\Desktop\terraform>terraform validate
Success! The configuration is valid.

```
C:\Users\student\Desktop\terraform>terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # github_repository.example will be created
  + resource "github_repository" "example" {
      + allow_auto_merge            = false
      + allow_merge_commit          = true
      + allow_rebase_merge          = true
      + allow_squash_merge          = true
      + archived                    = false
      + default_branch              = (known after apply)
      + delete_branch_on_merge      = false
      + description                 = "My awesome codebase"
      + etag                        = (known after apply)
      + full_name                   = (known after apply)
      + git_clone_url               = (known after apply)
      + html_url                    = (known after apply)
      + http_clone_url              = (known after apply)
      + id                          = (known after apply)
      + merge_commit_message        = "PR_TITLE"
      + merge_commit_title          = "MERGE_MESSAGE"
      + name                        = "demo_rep"
      + node_id                     = (known after apply)
      + primary_language            = (known after apply)
      + private                     = (known after apply)
      + repo_id                     = (known after apply)
      + squash_merge_commit_message = "COMMIT_MESSAGES"
      + squash_merge_commit_title   = "COMMIT_OR_PR_TITLE"
      + ssh_clone_url               = (known after apply)
      + svn_url                     = (known after apply)
      + topics                      = (known after apply)
      + visibility                  = "public"
      + web_commit_signoff_required = false

      + security_and_analysis (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

github_repository.example: Creating...
github_repository.example: Creation complete after 7s [id=demo_rep]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```




```
C:\Users\student\Desktop\terraform>terraform destroy
github_repository.example: Refreshing state... [id=demo_rep]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

  # github_repository.example will be destroyed
  - resource "github_repository" "example" {
      - allow_auto_merge            = false -> null
      - allow_merge_commit          = true -> null
      - allow_rebase_merge          = true -> null
      - allow_squash_merge          = true -> null
      - allow_update_branch         = false -> null
      - archived                    = false -> null
      - default_branch              = "main" -> null
      - delete_branch_on_merge      = false -> null
      - description                 = "My awesome codebase" -> null
      - etag                        = "W/\"37983761d160ab384633e69fc70bf566a5b68879bf72f9ee543e48f4c8171218\"" -> null
      - full_name                   = "Juhita2005/demo_rep" -> null
      - git_clone_url               = "git://github.com/Juhita2005/demo_rep.git" -> null
      - has_discussions             = false -> null
      - has_downloads               = false -> null
      - has_issues                  = false -> null
      - has_projects                = false -> null
      - has_wiki                    = false -> null
      - html_url                    = "https://github.com/Juhita2005/demo_rep" -> null
      - http_clone_url              = "https://github.com/Juhita2005/demo_rep.git" -> null
      - id                          = "demo_rep" -> null
      - is_template                 = false -> null
      - merge_commit_message        = "PR_TITLE" -> null
      - merge_commit_title          = "MERGE_MESSAGE" -> null
      - name                        = "demo_rep" -> null
      - node_id                     = "R_kgDOPdLjmQ" -> null
      - private                     = false -> null
      - repo_id                     = 1037231001 -> null
      - squash_merge_commit_message = "COMMIT_MESSAGES" -> null
      - squash_merge_commit_title   = "COMMIT_OR_PR_TITLE" -> null
      - ssh_clone_url               = "git@github.com:Juhita2005/demo_rep.git" -> null
      - svn_url                     = "https://github.com/Juhita2005/demo_rep" -> null
      - topics                      = [] -> null
      - visibility                  = "public" -> null
      - vulnerability_alerts        = false -> null
      - web_commit_signoff_required = false -> null
        # (2 unchanged attributes hidden)

      - security_and_analysis {
          - secret_scanning {
              - status = "enabled" -> null
```

```
          - secret_scanning_push_protection {
              - status = "enabled" -> null
            }
        }
    }

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

github_repository.example: Destroying... [id=demo_rep]
github_repository.example: Destruction complete after 0s

Destroy complete! Resources: 1 destroyed.
```

Conclusion :

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. It allows users to define and provision infrastructure resources in a declarative manner using configuration files.